TECHNICAL GAME DESIGN DOCUMENT

Project: "Start Reactor" Memory Minigame Architecture & Logic Blueprint

This technical design document outlines the comprehensive logic, engine-level implementation rules, and edge-case handling for the "Start Reactor" minigame. The goal is to provide a complete engineering blueprint for a highly responsive, lag-resistant, and bug-free recreation of the classic memory matching task.

1. Visual Layout & Mechanics

The interface is strictly divided into a two-panel, side-by-side layout. This dual structure separates information delivery (the pattern) from physical interaction (the input), requiring the player's eyes to track back and forth.

1.1 Interface Panels

- **The Display Panel (Left):** A read-only screen featuring a 3x3 grid of solid squares. These squares flash a bright blue color to demonstrate the pattern. Above this grid sits a row of five indicator lights tracking the current active stage.
- **The Input Panel (Right):** An interactive 3x3 keypad consisting of physical-looking buttons. Above this grid sits a secondary row of five indicator lights that track the player's step-by-step progress within the current stage.

1.2 Mathematical Grid Indexing

Both the Display Panel and the Input Panel map directly to identical zero-indexed, 1D arrays operating as a 2D matrix. Visual rendering and internal logic both process grid squares strictly as integers from `0` to `8`. Reading order is left-to-right, top-to-bottom.



Index Mapping Architecture

By mapping the layout to integer indices, visual logic is unified. A sequence requiring the top-right button, center button, and bottom-left button is stored purely as `[2, 4, 6]`.

When the logic commands index `4` to flash, it triggers the center square on the left screen. When the player clicks index `4` on the right keypad, it is instantly evaluated against the target array.

> **Design Constraint: Aspect Ratio Locking**
>
> To preserve muscle memory across devices, the 3x3 grids must be aspect-ratio locked (1:1). Stretched UI elements skew the spatial relation between buttons, which negatively impacts gameplay on mobile versus ultra-wide desktop monitors.

2. Game Progression & Pattern Generation

2.1 The Cumulative Stage Model

The game consists of five distinct, cumulative stages. The pattern does not change between stages; it strictly appends one new input to the end of the existing sequence.

- **Stage 1:** 1 total flash. Player inputs 1 button.
- **Stage 2:** Stage 1's flash + 1 new flash. Player inputs 2 buttons.
- **Stage 3:** Stage 2's flashes + 1 new flash. Player inputs 3 buttons.
- **Stage 4:** Stage 3's flashes + 1 new flash. Player inputs 4 buttons.
- **Stage 5:** Stage 4's flashes + 1 final flash. Player inputs 5 buttons.

2.2 Pre-Generation Algorithm

A common anti-pattern is generating the new random number only when a stage begins. This risks sync issues and makes resets difficult. Instead, the game utilizes a **Pre-Generation Algorithm**. The entire 5-step sequence is generated the exact millisecond the game initializes or is reset from a failure.

```
// Pseudocode for Global Sequence Generation
const MAX_STAGES = 5;
let targetSequence = [];
let currentStage = 1;

function generatePattern() {
    targetSequence = []; // Clear existing array
    for (let i = 0; i < MAX_STAGES; i++) {
        // Generate random integer between 0 and 8 inclusive
        let randomSquare = Math.floor(Math.random() * 9);
        targetSequence.push(randomSquare);
    }
}
// Expected output example: [4, 8, 8, 1, 3]
```

2.3 Duplicate Logic (Consecutive Flashes)

True randomness is heavily encouraged. The generation logic strictly omits any checks preventing duplicate consecutive numbers. It is entirely valid for the sequence to be `[4, 4, 4, 4, 4]`.

This is why visual implementation (Section 3) requires a distinct gap duration between flashes. Without a gap, consecutive duplicates would appear as a single, extra-long flash, completely obfuscating the pattern length to the player.

3. Core Game Loop & Asynchronous Timing

3.1 Initial Observation Delay

Upon instantiating the UI panel, the system must enforce a `1000ms` initialization buffer. If Stage 1 flashes instantaneously upon UI render, human reaction times dictate the player will miss it while their visual focus adjusts from the 3D game world to the 2D panel overlay. No logic or flashing occurs during this buffer.

3.2 The Show Pattern Phase

The playback logic cannot run synchronously. A standard loop would execute instantly, causing all target squares to flash on the same frame. Asynchronous execution is mandatory.

- **Flash Duration:** `400ms`. The square is illuminated in its active state (e.g., solid blue, or lit sprite).
- **Gap Duration:** `150ms`. The square returns to its dark state before the loop proceeds to the next index.

```
// Asynchronous Playback Loop
async function playSequence() {
    // Hard-lock the input panel during playback
    setInputState(LOCKED);

    for (let i = 0; i < currentStage; i++) {
        let activeIndex = targetSequence[i];

        illuminateDisplaySquare(activeIndex);
        await delay(400); // Wait for flash duration

        darkenDisplaySquare(activeIndex);
        await delay(150); // Mandatory gap
    }

    // Unlock input panel for the player
    setInputState(UNLOCKED);
    startInputTimeoutTimer();
}
```

3.3 Input Blocking

As demonstrated in the pseudocode above, physical input must be categorically rejected during the `playSequence()` execution. Any interaction with the right keypad while the left screen is actively flashing must be swallowed by the logic controller to prevent state desynchronization.

> ### Audio Syncing
>
> To enhance memorization, a specific tone is tied to each of the 9 grid indexes. The audio clip triggers identically when the square flashes on the display panel, AND when the player manually clicks the corresponding keypad button.

4. Technical Architecture & Engine Implementation

4.1 Client-Side Authority (Netcode)

To prevent network latency from destroying the tight timing windows of the memory game, this architecture employs strict **100% Client-Side Authority**.

No sequence generation, input validation, or state tracking occurs on the server. The local client handles the entire minigame natively. The client only dispatches an RPC (Remote Procedure Call) network event to the master server upon transitioning to the Victory State, notifying the server to increment the global task bar. If the client disconnects mid-task, the state is simply dropped.

4.2 Finite State Machine (FSM)

Loose boolean flags (e.g., `isFlashing`, `canClick`) create brittle logic prone to race conditions. The minigame is architected around a strict Finite State Machine. The system can only occupy one state at a time, and transitions follow rigid pathways.

FSM State	Description & Behavior
`STATE_INIT`	UI opened. Enforcing 1000ms Observation Delay. Keypad locked.
`STATE_SHOWING_PATTERN`	Async sequence playback active. Keypad locked. Timers paused.
`STATE_WAITING_INPUT`	Playback complete. Keypad unlocked. 15-second timeout timer ticking.
`STATE_EVALUATING`	Micro-state triggering the exact frame an input is received to validate it.
`STATE_ERROR`	Input failed or timed out. Red flash animation playing. Preparing to reset to Stage 1.
`STATE_VICTORY`	Stage 5 complete. Green complete animation playing. Server RPC fired.
`STATE_ABORTED`	Task interrupted. Destroying loops, garbage collecting.

4.3 Engine-Level Timing (Delta Time vs. Timers)

When implementing in a compiled engine (Unity, Unreal, Godot), relying on raw OS timers (like web `setTimeout`) or frame-counting leads to hardware desync. A 144Hz monitor will run frame-based logic faster than a 60Hz mobile display.

All async operations must rely on Engine Coroutines tracking **Delta Time** (the time elapsed since the last rendered frame). This guarantees a 400ms flash is mathematically verified against the engine clock, keeping the rhythm identical across all hardware.

4.4 Visual Rendering (Sprite-Swapping)

CSS-style color transitions or dynamic material instancing are overly expensive for this specific mechanic. The visual flash is achieved through instantaneous **Sprite Swapping**. The engine loads `button_idle.png` and `button_lit.png` into memory. During the flash sequence, the renderer simply updates the texture pointer for that specific UI element, costing near-zero GPU overhead.

4.5 Gamepad 2D Navigation Matrix

To support controller/keyboard input natively without virtual cursors, a 2D coordinate matrix overlays the 1D index array. A virtual selection highlight tracks `[x, y]` coordinates.

```
// Controller Matrix Navigation
let cursor = { x: 0, y: 0 };

function onDPadInput(direction) {
    if (direction === 'RIGHT' && cursor.x < 2) cursor.x++;
    if (direction === 'LEFT' && cursor.x > 0) cursor.x--;
    if (direction === 'DOWN' && cursor.y < 2) cursor.y++;
    if (direction === 'UP' && cursor.y > 0) cursor.y--;

    // Map 2D coordinate back to 1D array index
    let activeIndex = (cursor.y * 3) + cursor.x;
    highlightKeypadButton(activeIndex);
}
```

Pressing the primary controller face button (e.g., 'A' on Xbox) fires an event targeting `activeIndex` exactly as a mouse click would.

5. Advanced Input Validation & Edge Case Rules

6.1 Stage Transition Rules

Successful completion of a stage does not immediately begin the next pattern playback. Immediate transitions feel jarring and rob the player of psychological reward. Upon entering the final correct input for the current stage:

1. Illuminate the corresponding stage progress indicator (Top left).
2. Play a short success confirmation sound (Pitch-shifted chime).
3. **Wait approximately 500 milliseconds.**
4. Increment `currentStage`.
5. Begin playback of the next sequence.

Example: Stage 2 completed → Indicator 2 lights up → 500ms delay → Stage 3 sequence begins.

6.2 Input Validation Algorithm

Every keypad interaction is captured in `STATE_EVALUATING` and checked against the target array using the current step count. Correct inputs advance progress, incorrect inputs immediately trigger failure, and full step completion advances the stage (or triggers victory).

```
function validateInput(buttonIndex) {
    if (buttonIndex !== targetSequence[playerStep]) {
        enterErrorState();
        return;
    }

    playerStep++;

    if (playerStep === currentStage) {
        if (currentStage === MAX_STAGES) {
            enterVictoryState();
        } else {
            enterStageTransition(); // Handles 6.1 Logic
        }
    }
}
```

6.3 Input Timeout

The game must not remain indefinitely in `STATE_WAITING_INPUT`. A strict timer begins immediately after pattern playback completes. If the player is idle, the task resets.

- **Timeout Value:** `15 seconds`.
- **Failure Flow:** Timeout → `STATE_ERROR` → Reset to Stage 1 → Generate new sequence.

6.4 Input Feedback

Player inputs must provide immediate audiovisual confirmation, utterly independent of whether the input is correct or incorrect. Depriving the player of tactile feedback on a wrong guess makes the UI feel unresponsive. Upon button press: Illuminate the button, play a tone, register the input, and return the button to idle after ~100ms.

6.5 Simultaneous Input Protection

Hardware touchscreens and macro-enabled mice can submit multiple inputs on the same frame. The system must reject multi-touch and overlapping inputs.

- Only one active input may be processed per logic tick.
- Additional touch events arriving during `STATE_EVALUATING` are ignored.
- A rigid **50ms debounce window** is applied per button to prevent physical switch bounce or rapid-fire jitter from registering as two distinct clicks.

6.6 Input Queue Prevention

Players often memorize the pattern and rapid-fire input the sequence. Immediately upon receiving the final valid input, the state shifts out of `STATE_WAITING_INPUT`. All subsequent physical presses queued in the buffer MUST be discarded.

> **Input Queue Example**
>
> **Required sequence:** 4 → 0 → 8
>
> **Player rapidly enters:** 4 → 0 → 8 → 2 → 6 → 1
>
> *Result:* The inputs '2', '6', and '1' are safely discarded because the FSM transitioned the millisecond '8' was processed. They do not carry over to the next stage.

6.7 Fresh Press Requirement

A button physically held down during `STATE_SHOWING_PATTERN` must not automatically trigger an input when the game enters `STATE_WAITING_INPUT`. Valid input requires a discrete **Press Down** → **Release** event cycle after the input phase begins. Held inputs bridging state changes are ignored.

6.8 Stage Indicator Behavior

The five top-left indicator lights strictly display global stage progression. They reflect the state of stages 1 through 5 across the entire task lifecycle.

- **Off (Dark):** Stage not yet reached.
- **Blue:** Current active stage (Flashing).
- **Green:** Successfully completed stage.

6.9 Player Progress Indicator Behavior

The secondary five indicator lights (top-right, above keypad) display micro-progress within the active stage. Unlike the stage indicators, these are ephemeral.

Upon completing a stage, **all player indicators reset to Off** before beginning the next sequence playback. As the player successfully inputs steps, the lights turn green sequentially from left to right.

Stage 3 Execution Example	
Start of Stage 3 Input Phase	[Off] [Off] [Off] [Off] [Off]
Correct Input 1 Received	[Green] [Off] [Off] [Off] [Off]
Correct Input 2 Received	[Green] [Green] [Off] [Off] [Off]
Correct Input 3 Received	[Green] [Green] [Green] [Off] [Off] (Triggers Stage Success)
Start of Stage 4 Input Phase	[Off] [Off] [Off] [Off] [Off] (Complete Reset)

6.10 Abort State & Memory Leaks

Because the game utilizes asynchronous loops (Section 3.2), abruptly closing the UI without handling the loops will cause severe memory leaks and background null-reference exceptions. The `STATE_ABORTED` handler is mandatory.

Trigger Events: Manual UI close button, Emergency meeting called, Dead body reported, Level scene change, or network disconnect.

Teardown Execution:

1. Instantly cancel all active coroutines and clear all timeouts/intervals.
2. Remove UI event listeners.
3. Reset all visual indicators to Off.
4. Nullify the `targetSequence` array.
5. Destroy the UI prefab layer.

Re-opening the task after an abort always generates a completely new Stage 1 sequence, preventing players from "saving" progress on difficult patterns by closing the UI.

/// END OF DESIGN DOCUMENT ///